

Prompt Engineering in Generative AI

Prompt Engineering is the process of designing and optimizing instructions (prompts) given to an LLM (Large Language Model) so that it generates accurate, relevant, and useful outputs.

Just like a human performs better when given clear instructions, an LLM performs better when given well-structured prompts.

Prompting is critical in Generative AI because it dictates the quality, relevance, and accuracy of the output

Example

 **Bad Prompt:**

Write about AI.

Output may be vague and generic.

 **Better Prompt:**

Explain Artificial Intelligence to a 10-year-old child in 100 words with an example.

Output becomes focused and useful.

2. Why Prompt Engineering is Used

Prompt Engineering is important because LLMs do not truly "understand" human intentions.

They predict the next token based on patterns learned during training.

A good prompt helps the model understand:

- What task to perform
- What format to follow
- What level of detail is required
- What role it should play

Benefits of Prompt Engineering

1. Better Accuracy

Without guidance:

Summarize this document.

With guidance:

Summarize this document in 5 bullet points focusing only on financial risks.

The second prompt produces more relevant output.

2. Reduces Hallucinations

Hallucination means the model invents information.

Prompt:

If the answer is unknown, say "I don't know".

This reduces false information.

3. Produces Structured Outputs

Instead of:

Give customer information.

Use:

Return customer information in JSON format.

Output:

```
{  
  "name": "John",  
  "age": 25  
}
```

Components of a Good Prompt (RTCCO)

A strong prompt usually contains:

1. Role

Act as a Data Scientist.

2. Task

Analyze the dataset.

3. Context

The dataset contains customer purchase records.

4. Constraints

Limit response to 100 words.

5. Output Format

Return results as JSON.

Complete Example

Act as a Data Scientist.

Analyze the following customer data.

Identify churn risk.

Provide output in JSON format.

Limit explanation to 50 words.

This is a well-engineered prompt.

TYPES OF PROMPTING TECHNIQUES

1. Zero-Shot Prompting
2. One-Shot Prompting
3. Few-Shot Prompting
4. Chain-of-Thoughts
5. ReAct
6. Structured Output

1. Zero-Shot Prompting

Zero-shot prompting is a prompting technique in generative AI where you instruct a model to perform a task **without providing any task-specific examples**.

The model **relies entirely on its pre-trained knowledge** to infer the correct response.

Formula

Task Instruction + Input

Why It Is Used

Zero-shot prompting is used when:

- The task is simple and well-known.
- You don't want to provide examples.
- You need quick responses.

Examples

Translation

Prompt:

Translate "Hello" into French.

Output:

Bonjour

No examples were given.

How It Works

Suppose the prompt is:

Classify the sentiment.

Text: This movie was amazing.

Step 1: Prompt enters LLM

Classify sentiment

Text: This movie was amazing

Step 2: Model recognizes task

During pretraining, the model has seen:

- Sentiment analysis
- Reviews
- Positive words
- Negative words

So it understands:

Task = Classification

Step 3: Pattern Matching

The word:

amazing

is strongly associated with positive sentiment.

Step 4: Generate Output

Positive

No examples are required.

Advantages

1. Simple

Only instruction needed.

2. Fast

Short prompts.

3. Low Cost

Consumes fewer tokens.

4. Easy to Implement

No example preparation required.

Limitations

1. Less Accurate

For complex tasks.

2. Output Format May Vary

The model may not always follow the desired structure.

3. Higher Hallucination Risk

Compared to Few-Shot prompting.

Python Code with langchain example:

```
from langchain_groq import ChatGroq
from langchain.prompts import PromptTemplate

llm = ChatGroq(
    api_key="gsk_XXXXXXXXXXXXXXXXXXXX",
    model="llama-3.3-70b-versatile",
    temperature=0
)

template = """
Classify the sentiment as Positive, Negative, or Neutral.

Review:
{review}
"""

prompt = PromptTemplate(
    input_variables=["review"],
    template=template
)

chain = prompt | llm

result = chain.invoke({
    "review": "This product exceeded my expectations."
})

print(result.content)
```

